

2

0

Features

2026-10-28 10:38:58 p. m.

Tags | Timeline

Feature	Title	Passed	Failed	Scenarios	Time (ms)
billetera/features/flujo-administrador.feature	Flujo Administrador - Auditoría de la transferencia	1	0	1	8866
billetera/features/regresion.feature	Regresion - Billetera Digital	3	0	3	3974



3

0

Scenarios

2026-10-28 10:38:48 p. m.

[1:28] Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado

[2:68] Escenario 2 - Transferencia del usuario Emisor al Receptor

[3:118] Escenario 3 - Validación de la transferencia en el Receptor

Summary | Tags | Feature: ***billetera/features/regresion.feature*** | **Regresion - Billetera Digital**

Flujo de regresión end-to-end que genera su propia data de prueba: registra un usuario Emisor y un Receptor, les asigna saldo, transfiere 450.00 del Emisor al Receptor y valida saldos e historial de ambos. Los escenarios comparten datos mediante variables globales (utils.VariablesGlobales) y se ejecutan en orden dentro de una sola ejecución.

regresion parallel=false registro ms: 2433

Scenario: [1:28] **Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado**

```
>>  Background:
    # 1. Generar data de prueba con celulares aleatorios
30  * def emisor = crearUsuario('Emisor', generarCelular())                    17
31  * def receptor = crearUsuario('Receptor', generarCelular())                3
32  * match emisor.phone == '#regex 9[0-9]{8}'                                 30
33  * match receptor.phone == '#regex 9[0-9]{8}'                               1
34  * match emisor.phone != receptor.phone                                     1
    # 1.1 Registrar ambos usuarios (si alguno ya existe, el escenario se cae con error)
37  * def registroEmisor = call registrarUsuario { usuario: '#(emisor)' }      427
>>    billetera.helpers.registrar-usuario                                      415
38  * def registroReceptor = call registrarUsuario { usuario: '#(receptor)' }  216
>>    billetera.helpers.registrar-usuario                                      208
39  * set emisor.id = registroEmisor.usuarioId                                  3
40  * set receptor.id = registroReceptor.usuarioId                              1
41  * match emisor.id != receptor.id                                           1
    # 2. Asignar saldo suficiente a cada usuario y validar que se asignó correctamente
44  * def sesionEmisor = call login { phone: '#(emisor.phone)', pin: '#(emisor.pin)' }  201
>>    billetera.helpers.login                                                  195
45  * def recargaEmisor = call recargarSaldo { token: '#(sesionEmisor.token)', monto: '#
      (montoRecargaEmisor)' }                                                  667
>>    billetera.helpers.recargar-saldo                                         659
46  * match recargaEmisor.saldoNuevo == recargaEmisor.saldoAnterior +          1
      montoRecargaEmisor
47  * match recargaEmisor.saldoConsultado == recargaEmisor.saldoNuevo          1
48  * assert recargaEmisor.saldoConsultado >= montoTransferencia               2
50  * def sesionReceptor = call login { phone: '#(receptor.phone)', pin: '#(receptor.pin)' }  200
>>    billetera.helpers.login                                                  193
51  * def recargaReceptor = call recargarSaldo { token: '#(sesionReceptor.token)', monto: '#
      (montoRecargaReceptor)' }                                                630
>>    billetera.helpers.recargar-saldo                                         624
52  * match recargaReceptor.saldoNuevo == recargaReceptor.saldoAnterior +      1
      montoRecargaReceptor
53  * match recargaReceptor.saldoConsultado == recargaReceptor.saldoNuevo      1
    # 3. Guardar los saldos actuales de cada usuario en variables
56  * def saldoInicialEmisor = recargaEmisor.saldoConsultado                    0
57  * def saldoInicialReceptor = recargaReceptor.saldoConsultado                0
58  * set emisor.saldoInicial = saldoInicialEmisor                              0
59  * set receptor.saldoInicial = saldoInicialReceptor                          0
    # Publicar usuarios y saldos como variables globales para los siguientes escenarios
62  * Globales.guardar('emisor', emisor)                                        10
63  * Globales.guardar('receptor', receptor)                                    1
64  * karate.log('EMISOR ->', emisor.phone, '| id:', emisor.id, '| saldo inicial:', saldoInicialEmisor)  1
65  * karate.log('RECEPTOR ->', receptor.phone, '| id:', receptor.id, '| saldo inicial:',
      saldoInicialReceptor)                                                    1
```

transferencia regresion parallel=false ms: 939

Scenario: [2:68] **Escenario 2 - Transferencia del usuario Emisor al Receptor**

```
>>  Background:
69  * def emisor = obtenerGlobal('emisor')                                     1
70  * def receptor = obtenerGlobal('receptor')                                 1
```



3

0

Scenarios

2026-10-28 10:38:48 p. m.

[1:28] Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado

[2:68] Escenario 2 - Transferencia del usuario Emisor al Receptor

[3:118] Escenario 3 - Validación de la transferencia en el Receptor

```
    # 1. Login del Emisor y guardar el token en una variable
 73 * def sesionEmisor = call login { phone: '#(emisor.phone)', pin: '#(emisor.pin)' }   207
>>      billetera.helpers.login                                                          203
 74 * def tokenEmisor = sesionEmisor.token                                               1
 75 * match sesionEmisor.usuario.id == emisor.id                                         1
    # Categoría de gasto para clasificar el pago
 78 Given url walletUrl                                                                  0
 79 And path 'categories'                                                                0
 80 And header Authorization = 'Bearer ' + tokenEmisor                                   0
 81 When method get                                                                     196
 82 Then status 200                                                                      0
 83 And match response == '#[_ > 0]'                                                     5
 84 * def categoriasGasto = karate.filter(response, function(c){ return c.type == 'EXPENSE' })   4
 85 * match categoriasGasto == '#[_ > 0]'                                               1
 86 * def categoria = categoriasGasto[0]                                                1
    # 2. Transferencia de 450.00 con la nota "Pago de Servicios"
 89 Given url transactionUrl                                                             0
 90 And path 'transfer'                                                                  0
 91 And header Authorization = 'Bearer ' + tokenEmisor                                   0
 92 And request { recipientId: '#(receptor.id)', amount: '#(montoTransferencia)', description: '#
        (notaTransferencia)', categoryId: '#(categoria.id)' }                            2
 93 When method post                                                                    299
    # 3. Validar la transacción
 96 Then status 200                                                                      0
 97 And match response contains { transactionId: '#string', referenceCode: '#string', status:
        'COMPLETED', message: '#string' }                                                0
 98 And match response.referenceCode == '#regex REF-[0-9]+'                              0
    # 4. Guardar el código de la transacción en una variable (y como variable global)
101 * def transactionId = response.transactionId                                         0
102 * def codigoTransaccion = response.referenceCode                                    0
103 * def transaccion = { transactionId: '#(transactionId)', referenceCode: '#
    (codigoTransaccion)', monto: '#(montoTransferencia)', nota: '#(notaTransferencia)' }   1
104 * Globales.guardar('transaccion', transaccion)                                     0
105 * karate.log('TRANSACCION ->', transactionId, '| código:', codigoTransaccion)      1
    # 5. Consultar el saldo del Emisor
108 * def consultaEmisor = call consultarSaldo { token: '#(tokenEmisor)' }             196
>>      billetera.helpers.consultar-saldo                                              191
109 * def saldoActualEmisor = consultaEmisor.saldo                                     2
    # 6. Validar la disminución del saldo respecto al saldo inicial (Escenario 1, paso 3)
112 * def saldoEsperadoEmisor = emisor.saldoInicial - montoTransferencia               7
113 * match saldoActualEmisor == saldoEsperadoEmisor                                  1
114 * assert saldoActualEmisor < emisor.saldoInicial                                   1
115 * Globales.guardar('saldoFinalEmisor', saldoActualEmisor)                          2
```

regresion | parallel=false | validacion-receptor — ms: 601

Scenario: [3:118] **Escenario 3 - Validación de la transferencia en el Receptor**

```
>> Background:
119 * def receptor = obtenerGlobal('receptor')                                        0
120 * def transaccion = obtenerGlobal('transaccion')                                  1
    # 1. Login del Receptor y guardar el token en una variable
123 * def sesionReceptor = call login { phone: '#(receptor.phone)', pin: '#(receptor.pin)' }   201
>>      billetera.helpers.login                                                        197
124 * def tokenReceptor = sesionReceptor.token                                        0
125 * match sesionReceptor.usuario.id == receptor.id                                  0
    # 2. Consultar el saldo del Receptor
128 * def consultaReceptor = call consultarSaldo { token: '#(tokenReceptor)' }       194
>>      billetera.helpers.consultar-saldo                                            190
129 * def saldoActualReceptor = consultaReceptor.saldo                               1
```



3

0

Scenarios

2026-10-28 10:38:48 p. m.

[1:28] Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado

[2:68] Escenario 2 - Transferencia del usuario Emisor al Receptor

[3:118] Escenario 3 - Validación de la transferencia en el Receptor

	# 3. Validar el incremento respecto al saldo inicial (Escenario 1, paso 3)	
132	* def saldoEsperadoReceptor = receptor.saldoInicial + montoTransferencia	0
133	* match saldoActualReceptor == saldoEsperadoReceptor	0
134	* assert saldoActualReceptor > receptor.saldoInicial	0
135	* Globales.guardar('saldoFinalReceptor', saldoActualReceptor)	0
	# 4. Consultar las transacciones del Receptor	
138	Given url walletUrl	0
139	And path 'transactions'	0
140	And header Authorization = 'Bearer ' + tokenReceptor	0
141	When method get	191
142	Then status 200	0
143	And match response == '#[_ > 0]'	1
	# 5. Validar el número de transacción contra el del Escenario 2, paso 4	
146	* def movimientos = karate.filter(response, function(m){ return m.referenceCode == transaccion.referenceCode && m.type == 'INCOME' })	2
147	* match movimientos == '#[1]'	1
148	* def movimiento = movimientos[0]	0
149	* match movimiento.referenceCode == transaccion.referenceCode	0
150	* match movimiento.type == 'INCOME'	0
151	* match movimiento.amount == montoTransferencia	0



1

0

Scenarios

2026-10-28 10:38:58 p. m.

[1:25] Validar la transacción de la operación desde el Backoffice

Summary | Tags | Feature: ***billetera/features/flujo-administrador.feature*** | **Flujo Administrador - Auditoría de la transferencia** *Desde el Backoffice se valida la transacción realizada en el feature de Regresión, usando las variables globales publicadas por sus escenarios.*

administrador parallel=false auditoria ms: 8866

Scenario: [1:25] **Validar la transacción de la operación desde el Backoffice**

>>	Background:	
	# Login del administrador	
27	Given url adminUrl	0
28	And path 'login'	0
29	And request credencialesAdmin	0
30	When method post	199
31	Then status 200	0
32	And match response contains { token: '#string', type: 'Bearer' }	0
33	* def tokenAdmin = response.token	0
	# Auditoría de transacciones: la operación debe existir con sus datos correctos	
36	Given url adminUrl	0
37	And path 'transactions'	0
38	And header Authorization = 'Bearer ' + tokenAdmin	0
39	When method get	1166
40	Then status 200	0
41	And match response == '#[_ > 0]'	1
42	* def encontradas = karate.filter(response, function(t){ return t.referenceCode == transaccion.referenceCode || t.id == transaccion.transactionId })	32
43	* match encontradas == '#[_ > 0]'	1
44	* match each encontradas contains { amount: '#(transaccion.monto)' }	1
45	* def movEmisor = karate.filter(encontradas, function(t){ return t.userId == emisor.id || t.from == emisor.id })	1
46	* def movReceptor = karate.filter(encontradas, function(t){ return t.userId == receptor.id || t.to == receptor.id })	1
47	* match movEmisor == '#[_ > 0]'	0
48	* match movReceptor == '#[_ > 0]'	0
	# Los saldos vistos por el administrador deben coincidir con los de los usuarios	
51	Given url adminUrl	0
52	And path 'users'	0
53	And header Authorization = 'Bearer ' + tokenAdmin	0
54	When method get	7422
55	Then status 200	0
56	* def usuarioEmisor = karate.filter(response, function(u){ return u.id == emisor.id })[0]	19
57	* def usuarioReceptor = karate.filter(response, function(u){ return u.id == receptor.id })[0]	17
58	* match usuarioEmisor contains { status: 'ACTIVE', balance: '#(saldoFinalEmisor)' }	1
59	* match usuarioReceptor contains { status: 'ACTIVE', balance: '#(saldoFinalReceptor)' }	0